# SubBox Project Documentation

Oct 21, 2026 · Javeria Tahir

SubBox is a Flutter app for Android, iOS and web that tracks recurring subscriptions (streaming, software, gyms, cloud storage), what they cost and when they renew, so users avoid unwanted charges. All 5 modules of the Zynvex Solutions Internship (Batch 4, Mobile App Development) are complete, and the web app is live at subscription-box-app-9ad03.web.app.

## Modules

All five modules are delivered; Module 4 shipped 3 days after its Oct 18 deadline.

| # | Module | Delivered | Deadline |
|---|---|---|---|
| 1 | Project setup, Firebase integration and authentication | Email/password and Google sign-in, email verification, password reset, profile in Firestore | Sep 27, 2026 |
| 2 | Subscription management, categories and Firestore CRUD | Add/edit/remove, 7 categories, 11 fast-fill presets with logos, dashboard totals, real-time sync | Oct 4, 2026 |
| 3 | Payment tracking, renewals and notifications | Mark as paid, payment history, Deadlines tab with live countdowns, phone reminders | Oct 11, 2026 |
| 4 | Dashboard, expense analytics and smart insights | Analytics tab with charts, 12-month projection, smart insights, Alerts tab, CSV export | Oct 18, 2026 |
| 5 | Testing, security, UI/UX refinement and deployment | 39 tests, validated Firestore rules, account deletion, live web app, release APK | Oct 25, 2026 |

## Features

The app has five tabs: Dashboard, Deadlines, Add (+), Alerts and Analytics. Phones get a bottom bar, and web gets a sidebar.

| Area | What users can do |
|---|---|
| Accounts | Sign up with email (verified by link) or Google, reset a password, stay signed in, delete the account and all its data |
| Subscriptions | Track name, category, price, currency (USD, PKR, EUR, GBP), billing cycle, renewal date, free trial, payment method and cancel link. Fast-fill 11 popular services |
| Dashboard | Monthly and yearly totals, active count, renewals due in 3 days, search, category filter, 7-day runway, top insights, money saved |
| Deadlines | Renewals grouped as Critical (under 24h, live countdown), This week and Later. Mark paid or open the cancel page |
| Payments | Mark as paid records the payment and moves the renewal one cycle forward (month-end and leap-year safe). Full history per subscription |
| Alerts | Urgent items (due soon, overdue, price went up), scheduled reminders, payment history, reminder on/off and time of day |

| Area | What users can do |
|---|---|
| Reminders | Phone notifications 7, 3 or 1 day before or on the day, chosen per subscription |
| Analytics | Spending by category (donut), 12-month cash flow, last 6 months paid, biggest costs, currency switcher, CSV export |
| Smart insights | Flags trials converting, overdue renewals, price increases, yearly-billing savings, overlapping services and any subscription over 40% of spend |
| Savings | Removing a subscription as "I cancelled it" adds its yearly cost to Saved to date |

## Architecture

Screens read live data through Riverpod providers, which stream from Firestore. All calculations live in pure functions, so they can be tested without Firebase.

Data flow: Firebase Auth → `userProvider` → `SubscriptionService` (Firestore) → providers (`subscriptionsProvider`, `paymentsProvider`, `cancelledProvider`, `settingsProvider`) → screens. `main.dart` also re-schedules phone reminders whenever subscriptions or settings change.

| Layer | File | Responsibility |
|---|---|---|
| Entry | `lib/main.dart` | Firebase init, routing (Login → Verify email → Home), reminder sync |
| Model | `lib/models/subscription.dart` | Subscription, Payment, Cancellation, ReminderSettings; billing-cycle maths; formatting |
| Service | `lib/services/auth_service.dart` | Sign up/in, Google, verification, reset, account deletion |
| Service | `lib/services/subscription_service.dart` | Firestore CRUD, mark paid, payments, cancellations, settings, data deletion |
| Service | `lib/services/analytics.dart` | Spending by category, projections, insights, CSV (pure functions) |
| Service | `lib/services/notification_service.dart` | Plans and schedules local reminders (Android/iOS) |
| Screens | `lib/screens/` | Home (nav + dashboard), Deadlines, Alerts, Analytics, form, account sheet, auth screens |
| Widgets | `lib/widgets/` | Shared UI (cards, tags, stat grid) and hand-drawn charts (donut, bar, meter) |
| Theme | `lib/theme.dart` | Colors and type from the SubBox design system (dark slate, emerald) |

## Tech stack

| Area | Technology |
|---|---|
| Framework | Flutter 3.47 (Dart 3) |
| Authentication | Firebase Authentication (email/password, Google) |
| Database | Cloud Firestore (region asia-south1) |
| Hosting | Firebase Hosting |

| Area | Technology |
| --- | --- |
| State management | Riverpod 3 |
| Notifications | flutter_local_notifications + timezone |
| Fonts | Google Fonts (Hanken Grotesk, Geist) |
| Links | url_launcher |
| Charts | Hand-drawn with CustomPainter (no chart package) |
| Platforms | Android, iOS, web |

## Data model

Each user's data sits under their own document in Firestore; nothing is shared between users.

| Path | Fields | Notes |
| --- | --- | --- |
| `users/{uid}` | name, email, createdAt, reminderSettings (enabled, hour) | Created at sign-up |
| `users/{uid}/subscriptions/{id}` | name, category, price, currency, cycle, nextRenewal, isTrial, paymentMethod, cancelUrl, reminders, previousPrice | previousPrice is set when the price changes, for price-hike alerts |
| `users/{uid}/subscriptions/{id}/payments/{id}` | amount, currency, paidAt, forRenewal | Read-only record; deleted with its subscription |
| `users/{uid}/cancelled/{id}` | name, monthlyCost, currency, cancelledAt | Feeds Saved to date |

Monthly cost = price × charges per year ÷ 12 (weekly 52, monthly 12, quarterly 4, yearly 1). Amounts are never converted between currencies.

## Security

The validated Firestore rules (`firestore.rules`) were deployed on Oct 21, 2026, so the server rejects bad writes even from a modified app.

- **Access:** a user can read and write only their own data; everything else is denied by default.
- **Write validation:** only known fields are accepted, with types and limits (name ≤ 60 characters, price > 0 and < 10 million, known currencies and cycles, ≤ 4 reminders).
- **Payment records:** can be added or deleted, never edited.
- **Input checks in the app:** cancel links must be http(s), which blocks `javascript:` links. Full card numbers are rejected in Payment method. Prices allow at most 2 decimals.
- **CSV export:** cells starting with = + - @ are escaped, which prevents spreadsheet formula injection.
- **Account deletion:** removes all of the user's data, then the account. It requires a sign-in within the last 5 minutes and checks this before anything is deleted.
- **Web hosting headers:** HSTS, X-Frame-Options DENY, nosniff, strict referrer policy.

## Testing

All 39 automated tests pass, and `flutter analyze` reports no issues. Run them with `flutter test`.

| Test file | Tests | Covers |
|---|---|---|
| `test/analytics_test.dart` | 28 | Category spend, 12-month projection, paid-per-month history, 7-day runway, every insight rule, CSV escaping, reminder planning, input validators, charge dates |
| `test/widget_test.dart` | 6 | Monthly cost per cycle, days left, renewal after payment (month-end, leap year), logo matching, money and date formatting, auth validators |
| `test/screens_test.dart` | 5 | Analytics screen on phone and web sizes, currency switch, empty state, insight card tap |

Firebase calls (sign-in, Firestore reads and writes) are not covered by automated tests; check them by hand in the running app before each release.

## Setup and deployment

The web app is deployed to Firebase Hosting, and a release APK is built at `build/app/outputs/flutter-apk/app-release.apk`.

1. Install the Flutter SDK (stable channel) and the Firebase CLI, then run `firebase login`.
2. In the Firebase Console (project `subscription-box-app-9ad03`), enable Email/Password and Google sign-in and create the Firestore database.
3. For Android Google sign-in, add the SHA-1 fingerprint under Project settings and place `google-services.json` in `android/app/`.
4. Run locally: `flutter pub get`, then `flutter run` (pick an Android device or Chrome).
5. Check: `flutter analyze` and `flutter test`.
6. Deploy web and security rules: `flutter build web --release`, then `firebase deploy --only hosting,firestore:rules`.
7. Build Android: `flutter build apk --release`.

On Windows, `android/gradle.properties` sets `kotlin.incremental=false` because the project (D:) and the pub cache (C:) are on different drives.

## Known limitations and next steps

The app is ready for demos and a small group of testers; the first item below matters most for everyday use.

| Limitation | Effect | Next step |
|---|---|---|
| Renewal dates move only when the user taps Mark Paid | Auto-charged subscriptions show Overdue after their date, and no reminders are planned for the next cycle | Roll overdue dates forward automatically and log the charge; keep asking for free trials |
| Reminders are local to the phone and rescheduled only when the app opens | No reminders on web; long-idle phones miss new cycles | Server-side reminders with Cloud Functions (email or push), Blaze plan |
| No crash reporting | Crashes on users' phones go unseen | Add Firebase Crashlytics |

| Limitation | Effect | Next step |
| --- | --- | --- |
| No privacy policy | Required for the Play Store and for storing personal data | Publish a privacy policy page |
| App ID is `com.example.subbox_app` and the APK uses the debug key | Cannot be published on Google Play | New app ID, release signing key, its SHA-1 added in Firebase |
| 4 currencies, no conversion | Totals are shown per currency | Add currencies and optional conversion |
| Analytics re-reads all payments on every change | Fine for tens of users; heavier at scale | Cache or aggregate payment totals |
| iOS never built or tested | Unknown issues on iPhone | Build and test on iOS |